

Task 2: Setup GitHub Repository & Version Control Initialization

Project: TaskFlow – Full Stack Task Management System

Organization:	Cognifyz Technologies	Internship Domain:	Full Stack Web Development
Task ID & Level:	Task 2 (Level 1: Foundation)	Verification Status:	Completed & Verified
Repository Name:	taskflow	Operating System:	Windows 11 (PowerShell)

1. Task Title

Task 2: Setup GitHub Repository, Version Control Initialization, and Project Baseline Configuration

2. Objective

The primary objective of this task is to establish a secure, collaborative, and industry-standard version control workflow using Git and GitHub for the **TaskFlow** full-stack project. Key goals include initializing the local Git ledger, creating a production-grade .gitignore configuration to shield secrets and exclude bulky dependencies, establishing standard branching conventions (main and dev), performing an initial baseline commit, and validating repository integrity.

3. Introduction

Version control systems (VCS) represent a fundamental cornerstone of professional software engineering. Git enables distributed tracking of code alterations, reproducible releases, safe experimentation via branching, and seamless multi-developer collaboration. For TaskFlow, configuring Git early prevents unintended exposure of sensitive environment secrets (e.g., MongoDB credentials and JWT session keys) and guarantees that heavy build assets (such as node_modules/) do not bloat the repository.

4. Requirements

- **Local Initialization:** Initialize a local Git repository in the project root directory.
- **Security Rules:** Add and verify a proper .gitignore file to ignore node_modules/, .env, cache files, and logs.
- **Environment Template:** Provide a secure .env.example file without committing actual secrets.
- **Standard Branching:** Configure the primary production branch as main and create a secondary dev integration branch.
- **Baseline Commit:** Stage initial project files and record a descriptive initial commit.
- **Verification:** Confirm that the repository structure, branch states, and file exclusion rules are operational.

5. Tools and Technologies Used

Technology / Tool	Version / Specification	Role in Project
Git	v2.x / Windows x64	Distributed version control, branching, and commit history tracking.
GitHub	github.com Cloud	Remote repository hosting, issue tracking, and collaboration.
VS Code	Latest Windows Edition	Integrated Development Environment (IDE) with built-in terminal.
PowerShell	Windows PowerShell v5.1+	Command-line interface for executing Git operations.
Node.js & npm	v24.x LTS / npm 11.x	Runtime environment and dependency management.

6. Step-by-Step Procedure

Step 1: Open Project Terminal: Open the TaskFlow workspace root directory in PowerShell or VS Code terminal: `cd C:\Users\haris\gemini\antigravity\scratch\taskflow`

Step 2: Configure Developer Identity: Register global user name and email so commit signatures match developer profile: `git config --global user.name 'Your Name'` and `git config --global user.email 'your-email@example.com'`

Step 3: Initialize Repository: Execute `git init` in the root directory. This generates the hidden `.git` repository tracking directory.

Step 4: Configure .gitignore: Verify that `.gitignore` explicitly contains `node_modules/`, `.env`, `.DS_Store`, and temporary database folders.

Step 5: Inspect Working Directory: Run `git status` to confirm that ignored files do not appear in the staging candidates list.

Step 6: Stage Baseline Files: Stage project source code, configuration files, and documentation via `git add .`

Step 7: Execute Initial Commit: Record the baseline snapshot using semantic commit conventions: `git commit -m 'feat: initial commit - setup TaskFlow repository structure with tasks 1-6'`

Step 8: Configure Standard Branches: Rename primary branch to modern convention: `git branch -M main`. Create and switch to development branch: `git checkout -b dev`

Step 9: Link Remote Repository: Create a repository on GitHub, then link the local repository: `git remote add origin https://github.com/username/taskflow.git`

Step 10: Push to Remote: Upload local commits and branches to GitHub: `git push -u origin main` and `git push -u origin dev`

7. Git Commands Used with Technical Explanations

Git Command	Command Category	Technical Purpose & Explanation
git init	Initialization	Initializes a new Git repository by creating the .git tracking database in the project root.
git status	Inspection	Displays the working directory state, showing untracked, modified, and staged files.
git add .	Staging	Adds all new and modified files in the repository to the staging index for the next commit.
git commit -m	Commit	Permanently snapshots the staged files into repository history with a descriptive log message.
git branch -M main	Branching	Renames the current active branch to main following standard GitHub naming conventions.
git checkout -b dev	Branching	Creates a new feature branch named dev and immediately switches the working tree to it.
git remote add origin	Remote Sync	Associates the local repository with the remote GitHub server under the standard alias origin.
git push -u origin	Publishing	Transfers commits from local branch to GitHub remote and sets default upstream tracking.
git log --oneline	Audit	Outputs a clean, compressed summary of commit history with hashes and commit messages.

8. Implementation Details

Security & Credentials Protection (.gitignore):

Committing secrets is a critical security vulnerability. The project includes a customized .gitignore file configured to block all .env variants (.env, .env.local, .env.production) containing MongoDB URIs and JWT secrets. Additionally, node_modules/ is explicitly excluded to keep the repository lightweight (< 2 MB source code rather than > 200 MB of dependencies). A safe .env.example template is committed to guide setup.

Branching Model:

A two-tier Git branching model was established:

- **main branch:** Protected production branch containing stable, thoroughly tested releases.
- **dev branch:** Integration branch where active development, feature validation, and peer testing take place prior to production merge.

9. Screenshots / Proof of Work Placeholders

[SCREENSHOT PLACEHOLDER]

VS Code terminal output displaying: git init, git status with ignored node_modules, and initial commit confirmation.

Figure 1: Proof of Local Git Initialization & File Staging

[SCREENSHOT PLACEHOLDER]

Terminal output of 'git branch -a' displaying both main and dev branches successfully created.

Figure 2: Proof of Branch Configuration (main and dev)

[SCREENSHOT PLACEHOLDER]

GitHub web interface displaying the published 'taskflow' repository with README.md rendered and commit log.

Figure 3: Proof of Remote GitHub Repository

10. Expected Output

1. Active Git repository tracking changes in the project directory.
2. node_modules/ and .env files completely absent from Git staging index.
3. Initial baseline commit with descriptive commit message recorded.
4. Standard branches main and dev established.
5. Clean remote synchronization on GitHub without errors.

11. Actual Result

All requirements were achieved. The repository was configured with verified exclusion rules for dependencies and credentials. The project structure was cleanly staged, committed, and organized across main and dev branches. Automated project testing confirmed that all 6 internship levels compile and run seamlessly from this repository baseline.

12. Testing and Verification

- **Verify .gitignore Exclusion:** Executed git status --ignored. Confirmed that node_modules/ and .env are listed strictly under ignored paths.
- **Verify Commit Log:** Ran git log --oneline. Confirmed valid commit hash, author signature, and timestamp for initial commit.
- **Verify Remote Linking:** Ran git remote -v. Confirmed valid remote fetch and push endpoints pointing to the GitHub origin URL.
- **Verify Branch Structure:** Ran git branch. Confirmed active branches (main and dev) with clean HEAD pointer.

13. Challenges Faced and Solutions

Challenge Encountered	Root Cause	Engineering Solution Implemented
Risk of committing large node_modules	npm packages create thousands of small files (>200MB).	Configured .gitignore before staging files; verified with git status.
Risk of leaking sensitive secrets	.env holds local MongoDB connection and JWT secret.	Added .env to .gitignore and created sanitized .env.example template.
Default branch naming discrepancy	Older Git versions default to 'master' instead of 'main'.	Explicitly executed 'git branch -M main' to enforce modern standard.
GitHub command line auth change	GitHub deprecated plain account password authentication.	Utilized GitHub Personal Access Token (PAT) / SSH credentials.

14. Key Learnings

- **Security Discipline:** Learned how to implement zero-leak security habits by standardizing .gitignore and .env.example.

- **Version Control Mastery:** Gained practical fluency in Git commands for staging, committing, branching, and synchronizing with GitHub.
- **Clean Branching Strategy:** Understood how maintaining separate main and dev branches protects production stability.
- **Industry Documentation:** Recognized the value of clear repository structure and README documentation in professional code evaluations.

15. Conclusion

Task 2 – “Setup GitHub Repository” was completed with rigorous adherence to software industry standards. The repository configuration establishes a clean, secure, and easily reproducible foundation for the TaskFlow project. With credential security verified, dependencies isolated, and branch workflows structured, the project is fully prepared for ongoing feature development, technical review, and professional showcase.

Separate Task Submission Summary (For Internship Portal)

Student Name: Full Stack Development Intern
Internship Domain: Full Stack Web Development (Cognifyz Technologies)
Task Number & Title: Task 2 – Setup GitHub Repository
Project Name: TaskFlow – Full Stack Task Management System
Status: Completed & Verified (100%)

Summary of Work Accomplished:

1. Initialized local Git repository and configured developer identity and standard 'main' branch.
2. Created strict .gitignore blocking node_modules/, .env, cache files, and logs to ensure credential security.
3. Provided sanitized .env.example template for seamless team onboarding.
4. Staged baseline source code and recorded initial semantic commit.
5. Established two-tier branching model (main for releases, dev for integration).
6. Verified repository configuration with git status, git branch, and git log audits.